Capstone 7-C — Agent Evolution: UCC Risk Analyzer

Build the SAME UCC delinquency-risk agent THREE times — first by hand with the raw API, then with the Agent SDK and Claude Code, then from a spec document. Same tools, same mock data, same business question. Three radically different developer experiences.

A — Healthcare Pre-Auth | B — B2B Order Exception | C — UCC Risk Analyzer

Project Brief

THREE WAYS TO BUILD A HOUSE

Imagine you decide to build the same house three times. The first time, you fell every tree, mill every plank, and hammer every nail by hand. You learn exactly where the load-bearing walls go, why the joists are spaced 16 inches apart, and what happens when a sill plate is undersized. The build takes six months, but you understand every joint in the structure.

The second time, you order pre-cut lumber from a mill, pre-fab trusses, and use a nail gun. The framing crew shows up with a power tool for every job. You finish in six weeks. The walls go up in the same places, but you are no longer wondering *why* — you are picking which tools to use and where to apply them. You build twice as fast and the structure is just as strong, but only because you already know what a structure should look like.

The third time, you hand a contractor a set of architectural drawings. Two weeks later, the house is done — framing, plumbing, electrical, finishes — built by people you never met to specifications you wrote in plain English. You are now an architect. The previous two builds taught you what to draw and what to leave to the crew. Skip those, and your drawings would not survive contact with reality.

This capstone is those three builds, compressed into one week. You will write the same UCC risk agent in raw API code, then in the Agent SDK with Claude Code, then as a 12-section spec that Claude Code reads and implements. By the end, you will know in your hands — not just in theory — why each layer of abstraction exists, what it costs, and when to reach for each one.

WHAT YOU'LL BUILD

You build a UCC Filing Risk Analyzer agent that takes a business name and produces a delinquency risk narrative: it searches UCC filings (handling name variations, abbreviations, DBAs), aggregates filing statistics, runs an ML delinquency model, examines the highest-risk filings, and writes a narrative report citing specific evidence.

You build it three times:

- ITERATION 1 Raw API loop — `~250 lines`, `~3 hours`, hand-coded everything (M15B way)
- ITERATION 2 Agent SDK + Claude Code — `~120 lines`, `~2 hours` (M25–M26 way)
- ITERATION 3 Spec-driven — `~100 lines of spec`, `~1 hour` (production way)

WHY IT MATTERS

Production teams do not pick "raw API vs SDK vs spec" in the abstract — they pick based on what the team already understands and what the problem requires. If you skip Iteration 1, you cannot debug Iteration 3 when the generated code does something weird. If you skip Iteration 3, you are 10x slower than the teams shipping agents in 2026. The point of building the same thing three times is that the *differences* teach you the trade-offs in a way no diagram or article ever will. You are not learning three different agents. You are learning three different *levels of abstraction*, and gaining the judgment to know which one fits which problem.

Three Iterations at a Glance

Same agent, same business question, same five tools, same mock data. Three layers of abstraction.

	Iter 1 — Raw API	Iter 2 — Agent SDK + Claude Code	Iter 3 — Spec-Driven
LINES YOU WRITE	~250 lines of Python	~120 lines (SDK + hooks + sessions)	~100 lines of spec (no agent code)
TIME TO BUILD	~3 hours	~2 hours	~1 hour
LOOP LIVES IN	Your `while True`	SDK's `query()`	Generated for you by Claude Code
DEBUG METHOD	`print()` in the loop + manual message inspection	Hooks as probes + Anthropic Console + Langfuse traces	Spec ↔ code comparison + tests + evals
KEY INSIGHT	Builds the mental model — every line is yours	Half the code; modular debug; sessions + hooks for free	The spec IS the documentation auditors read

The Three-Iteration Concept

Each iteration produces a working agent that solves the SAME business problem with the SAME tools and SAME mock data. The agent's *output* is identical across all three. What changes is everything around the agent: the lines you wrote, the time you spent, the abstractions you used, and the way you debug when something breaks.

Think of it like this: an agent is "a system prompt + tools + a loop." Iteration 1 makes you build all three from scratch. Iteration 2 keeps the system prompt and tools the same, but the SDK runs the loop for you. Iteration 3 keeps everything the same, but Claude Code writes the prompt, tools, AND loop from a spec you gave it. The agent is unchanged. You changed.

> ***A COMMON MISUNDERSTANDING***
>
> ***"Iteration 3 is just better — why would I bother with the others?"*** *Because Iteration 3's generated code is not magic. When it produces a tool that searches case-sensitively and you wanted case-insensitive, you have to read the generated `tools.py`, find the bug, and fix it — either in the code or in the spec. Without Iteration 1's deep familiarity with what tool calls, message shapes, and stop reasons look like, you cannot tell what the generated code is doing wrong. Iteration 3 is fast precisely because you can read its output. Take away that ability and it becomes a black box that produces silently broken agents.*

The Scenario — UCC Filing Risk Analyzer

The agent takes a business name and produces a delinquency risk narrative. It searches UCC filings (with name variations including DBAs and abbreviations), aggregates filing statistics, runs an ML delinquency model, examines the highest-risk filings, and writes a narrative report citing specific evidence.

> **BUSINESS QUESTION (USE THIS IN ALL THREE ITERATIONS)**
>
> "Assess the delinquency risk for Acme Corporation using UCC filing data."

TOOLS (3)

- `search_filings(debtor_name, state?)` — partial-match UCC search, case-insensitive
- `get_filing_details(filing_number)` — full record lookup
- `predict_delinquency(features...)` — sklearn RandomForest pickle, returns probability + HIGH/MEDIUM/LOW

MOCK DATA SHAPE

- **9 filings for Acme Corporation** across NY (4), CA (2), TX (2), FL (1, DBA "ACME CORP DBA ROADRUNNER SUPPLIES")
- **2 filings for Pinnacle Industries**, **1 for Sunrise Holdings**

- **Trained pickle model** with 6 features (active count, state count, collateral types, age, amendment frequency, months-to-lapse)
- **Files:** `mock_data.py`, `delinquency_model.pkl`, `test_scenarios.json`

WHY THIS SCENARIO

You have already seen this domain in M09 (RAG), M12 (ReAct), M15B (build), CAPSTONE-3-DOMAIN-C (entity resolution), CAPSTONE-5-DOMAIN-C (production), and CAPSTONE-6 (parallel state testing). Picking this scenario lets you focus on the *iteration differences* instead of learning a new domain. If you'd rather work in a different industry, switch to Domain A (Healthcare) or Domain B (B2B Orders).

Architecture Per Iteration

Each iteration produces the same logical agent (system prompt + 5 tools + a loop). What changes is the *physical* architecture — how much you own, how much the SDK owns, and how much is generated for you.

ITER 1 **Raw API Loop** — *You own everything*

agent.py *~250 lines, all hand-written*

```
while True:  # the loop you wrote
client.messages.create(...)
if response.stop_reason == "end_turn": break
for block in response.content:
    validate_input · check_cost_cap · log · execute_tool · redact_phi · append · audit_log
```

tools.py	circuit_breaker.py	audit_log.jsonl
5 clinical tools you wrote	*you built*	*you rotate*

server.py (FastAPI) + Dockerfile — you wrote both

ITER 2 **Agent SDK + Claude Code** — *You own the tools and config; SDK owns the loop*

agent.py *~90 lines*

```
@tool("lookup_clinical_criteria", ...) ×5 functions
preauth_server = create_sdk_mcp_server(tools=[...])
OPTIONS = ClaudeAgentOptions(system_prompt=..., mcp_servers=..., hooks=...)
async for msg in query(prompt=..., options=OPTIONS): ...
```

The loop, message-passing, retries, and streaming live inside `claude-agent-sdk`. You do not write them.

Claude Code generated	claude-agent-sdk managed
`server.py · Dockerfile · tests · .claude/settings.json · hooks/*.py`	`query() loop · MCP transport · HookMatcher · resume tokens · streaming`

ITER 3 **Spec-Driven** *You own only the spec*

spec/agent-spec.md *~100 lines — the only file you write*

1. Overview
2. Configuration
3. Tools (5)
4. System Prompt
5. Hooks (PHI / tone / risk)
6. Sessions
7. Mock Data
8. API Wrapper
9. Deployment
10. Tests
11. Evaluation
12. File Structure

↓

`/generate-from-spec spec/agent-spec.md`

Claude Code reads the spec, generates ~18 files: `agent.py` · `sessions.py` · `mock_data/*.json` ×4 · `server.py` · `Dockerfile` · `.claude/settings.json` · `hooks/*.py` · `.claude/commands/*.md` · `tests/test_*.py` ×5 · `appendix/manual-loop.py` *(Iter-1 reference).*

Prerequisites

REQUIRED MODULES

- **M05 — Tool Use:** Tool definitions, tool_use blocks, the message loop
- **M12 — ReAct Agents:** Multi-step reasoning, the think→act→observe pattern
- **M15B — Build Complete Agent:** The whole Iter-1 mental model lives here
- **M16–M17 — Guardrails & HITL:** The hooks you'll wire up in Iter 2
- **M19 — Tracing (Langfuse):** Optional but useful for Iter 2 debugging
- **M21 — Deployment:** FastAPI + Docker patterns
- **M22B — Cloud Deployment:** Tier 1 deployment for all three iterations
- **M25 — Claude Code:** CLAUDE.md, slash commands, sessions
- **M26 — Hooks & Sessions:** Pre/post tool-use hooks, session forking

IF YOU HAVE NOT DONE M15B / M26

Iteration 1 is essentially a re-implementation of the M15B reference agent for the UCC scenario. If you have not built an agent from raw API calls before, do M15B first — otherwise Iteration 1 will be confusing rather than illuminating, and Iteration 3's debugging steps will not work because you will not recognize what the generated code is doing. Iteration 2 leans heavily on the `claude-agent-sdk` patterns taught in M26 (Hooks & Sessions & Agent SDK) — reach for it if any of the `@tool` / `HookMatcher` / `ClaudeAgentOptions` calls feel unfamiliar.

TOOLS YOU'LL NEED INSTALLED

- Python 3.10+ with pip
- scikit-learn (for the delinquency pickle model)
- Claude Code CLI (`npm i -g @anthropic-ai/claude-code`) — only needed for Iter 2 and Iter 3
- Docker Desktop (for the Tier 1 deployment in each iteration)
- An Anthropic API key (`ANTHROPIC_API_KEY` environment variable)
- Optional: a Langfuse account for Iter 2 tracing (free tier is fine)

SESSION 1

Iteration 1: Raw API Loop

Build the agent the M15B way. You write the loop, the validation, the logging, the redaction, the sessions, and the deployment. Every line is yours. Every bug is yours to find.

`~3 hours` `~250 lines` `7 files` `0 abstractions`

Step 1: Setup & Mock Data

`15 min` `mock_data.py` `delinquency_model.pkl`

What & Why: Create the project folder, install `anthropic` + `scikit-learn`, then build the mock data your agent will read. Mock data is what separates a "demo" agent from a "doesn't compile" agent — without it, every tool call returns an error and you cannot tell if the loop is broken or the data is. We also train a tiny RandomForest pickle so `predict_delinquency` has something real to call.

SETUP · **MOCK_DATA.PY**

SETUP

```
mkdir agent-iter1-raw && cd agent-iter1-raw
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "anthropic≥0.40" "scikit-learn≥1.3" "pandas≥2.0" "fastapi≥0.110" "uvicorn≥0.27"

# Train the delinquency_model.pkl
python -c "
FROM sklearn.ensemble import RandomForestClassifier
IMPORT pickle, numpy as np
np.random.seed(42)
X = np.random.rand(200, 6); y = (X.sum(axis=1) > 3).astype(int)
clf = RandomForestClassifier(n_estimators=20, random_state=42).fit(X, y)
pickle.dump(clf, open('delinquency_model.pkl', 'wb'))
print('Model saved')
"
```

MOCK_DATA.PY

```
"""mock_data.py — 12 UCC filings across 3 entities."""
FILINGS = [
    # Acme Corporation — 9 filings across 4 states (incl. 1 DBA)
    {"filing_number": "NY-2024-0847", "debtor_name": "ACME CORPORATION",
     "state": "NY", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-03-15", "lapse_date": "2029-03-15",
     "secured_party": "FIRST NATIONAL BANK",
     "collateral": "All inventory and accounts receivable"},
    {"filing_number": "NY-2024-0848", "debtor_name": "ACME CORPORATION",
     "state": "NY", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-04-22", "lapse_date": "2029-04-22",
     "secured_party": "JPMORGAN CHASE",
     "collateral": "Equipment and machinery"},
    {"filing_number": "NY-2024-0849", "debtor_name": "ACME CORP",
     "state": "NY", "filing_type": "UCC3_AMENDMENT", "status": "ACTIVE",
     "filing_date": "2024-06-01", "lapse_date": "2029-04-22",
     "secured_party": "JPMORGAN CHASE",
     "collateral": "Equipment, machinery, and titled vehicles"},
    {"filing_number": "NY-2024-0850", "debtor_name": "ACME CORPORATION INC",
     "state": "NY", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-08-10", "lapse_date": "2029-08-10",
     "secured_party": "WELLS FARGO",
     "collateral": "Accounts receivable"},
    {"filing_number": "CA-2024-1201", "debtor_name": "ACME CORP",
     "state": "CA", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-02-18", "lapse_date": "2029-02-18",
     "secured_party": "BANK OF AMERICA",
     "collateral": "All assets of debtor"},
    {"filing_number": "CA-2024-1202", "debtor_name": "ACME CORPORATION",
     "state": "CA", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-05-05", "lapse_date": "2029-05-05",
     "secured_party": "CITIBANK NA",
     "collateral": "Inventory"},
    {"filing_number": "TX-2024-0903", "debtor_name": "ACME CORP",
     "state": "TX", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-07-12", "lapse_date": "2029-07-12",
     "secured_party": "PNC FINANCIAL",
     "collateral": "Equipment"},
    {"filing_number": "TX-2024-0904", "debtor_name": "ACME CORPORATION",
     "state": "TX", "filing_type": "UCC3_CONTINUATION", "status": "ACTIVE",
     "filing_date": "2024-09-01", "lapse_date": "2034-07-12",
     "secured_party": "PNC FINANCIAL",
     "collateral": "Equipment"},
    {"filing_number": "FL-2024-0455",
     "debtor_name": "ACME CORP DBA ROADRUNNER SUPPLIES",
     "state": "FL", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-10-30", "lapse_date": "2029-10-30",
     "secured_party": "TRUIST FINANCIAL",
     "collateral": "Motor vehicles and titled goods"},
    # Pinnacle Industries — 2 filings
    {"filing_number": "NY-2024-0501", "debtor_name": "PINNACLE INDUSTRIES",
     "state": "NY", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2023-11-10", "lapse_date": "2028-11-10",
```

```
     "secured_party": "TD BANK", "collateral": "Equipment"},
    {"filing_number": "NJ-2024-0302", "debtor_name": "PINNACLE INDUSTRIES",
     "state": "NJ", "filing_type": "UCC1", "status": "TERMINATED",
     "filing_date": "2022-05-15", "lapse_date": "2027-05-15",
     "secured_party": "US BANCORP", "collateral": "Inventory"},
    # Sunrise Holdings — 1 filing
    {"filing_number": "CA-2024-1500", "debtor_name": "SUNRISE HOLDINGS LLC",
     "state": "CA", "filing_type": "UCC1", "status": "ACTIVE",
     "filing_date": "2024-01-05", "lapse_date": "2029-01-05",
     "secured_party": "FIRST NATIONAL BANK",
     "collateral": "All assets of debtor"},
]
```

Run: `python -c "from mock_data import FILINGS; print(len(FILINGS), 'filings loaded')"`

Expected output:

```
Model saved
12 filings loaded
```

Checkpoint

You should see `12 filings loaded` and have `delinquency_model.pkl` in the folder. If pickle write fails, check write permissions.

Step 2: Define Tools as JSON Schema

`15 min` `tools.py`

What & Why: The Anthropic API needs every tool described as a JSON Schema object so Claude knows what arguments to pass. You also need a Python function that *executes* the tool when Claude asks. Get the schema wrong and Claude either ignores the tool or passes the wrong types; get the executor wrong and Claude gets back errors instead of data and may either retry forever or give up early.

TOOLS.PY

TOOLS.PY

```
IMPORT pickle
FROM mock_data import FILINGS

TOOLS = [
    {
        "name": "search_filings",
        "description": "Search UCC filings by debtor name (partial, case-insensitive). Optional state filter.",
        "input_schema": {
            "type": "object",
            "properties": {
                "debtor_name": {"type": "string"},
                "state": {"type": "string", "description": "2-letter code (optional)"},
            },
            "required": ["debtor_name"],

    # … (full source in the desktop HTML) …

                  args["collateral_types"], args["filing_age_years"],
                  args["amendment_frequency"], args["months_to_lapse"]]]
        prob = float(_MODEL.predict_proba(feats)[0][1])
        RETURN {"probability": prob,
                "prediction": "HIGH" if prob > 0.66 else "MEDIUM" if prob > 0.33 else "LOW"}
    RAISE ValueError(f"Unknown tool: {name}")
```

Checkpoint

Run `python -c "from tools import execute_tool; print(len(execute_tool('search_filings', {'debtor_name': 'acme'})))"`. Expected: `9` (all Acme variants found via case-insensitive partial match, including the DBA).

Step 3: Build the While Loop

`45 min` `agent.py` `The CORE of Iter 1`

What & Why: This is the heart of the iteration — the agentic loop you will replace twice over in later iterations. Write it once by hand and you will recognize what the SDK and Claude Code generate later. Skip it and you will not be able to debug them.

The loop has three jobs in a fixed order: **(1)** send the running message history to `messages.create`, **(2)** inspect `response.stop_reason`, **(3)** if `tool_use`, execute the tool, append both the assistant turn and the `tool_result` turn to the messages list, and loop back. The single most common Iter-1 bug is appending the tool_use without the matching tool_result, which fails on the next turn with a confusing 400 error.

AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

AGENT.PY (PYTHON)

```python
IMPORT json, anthropic
FROM tools import TOOLS, execute_tool

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"
MAX_TURNS = 10
SYSTEM = """You are a UCC filing risk analyst. When given a business name, search
thoroughly using name variations including abbreviations and DBAs, gather filing
statistics, run the delinquency model, examine the riskiest filings, and write
a narrative report citing specific evidence."""

FUNCTION _run_messages(messages):

    FOR turn IN range(MAX_TURNS):

    # … (full source in the desktop HTML) …


    text, _ = _run_messages([{"role": "user", "content": question}])
    RETURN text

IF __name__ == "__main__":
    print(run_agent("Assess the delinquency risk for Acme Corporation."))
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — the raw API loop. Everything is hand-coded.
IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT { TOOLS, executeTool } from "./tools.js";

const client = NEW Anthropic();
const MODEL = "claude-sonnet-4-6";
const SYSTEM = `You are a UCC filing risk analyst. When given a business name,
search thoroughly using name variations including abbreviations and DBAs,
gather filing statistics, run the delinquency model, examine the riskiest
filings, and write a narrative report citing specific evidence.`;

export FUNCTION runAgent(question, maxTurns = 10): Promise<string> {
  type Msg = Anthropic.MessageParam;
  const messages= [{ role, content: question }];

    # … (full source in the desktop HTML) …

  throw NEW Error(`Agent exceeded ${maxTurns} turns without finishing`);
}

IF (import.meta.url === `file://${process.argv[1]}`) {
  console.log(runAgent("Assess the delinquency risk for Acme Corporation."));
}
```

Run: `python agent.py`

Expected output (paraphrased — Claude's exact wording varies):

```
Acme Corporation has 9 active UCC filings across 4 states (NY:4, CA:2, TX:2, FL:1).
The Florida filing uses the DBA "ACME CORP DBA ROADRUNNER SUPPLIES" — missed by
naive name search.

Risk score: 0.95 (HIGH, capped). Driven by 9 active filings spread across 4 states.
Recommend deeper review of CA-2024-1201 (all assets) and TX-2024-0904 (recent
UCC-3 continuation extending the lien another 5 years).
```

✅ **Checkpoint — Step 3**

You should see a narrative report mentioning multiple Acme filings across NY/CA/TX/FL, the DBA variant, a probability, and a HIGH risk verdict. The exact wording will vary each run; the *structure* is what matters: did the agent search, score, and write a report? If yes, Iter 1's core loop is working.

Troubleshooting

- **tool_use_id error** → the assistant turn must be appended to `messages` BEFORE the `tool_result` turn. Check the order in `_run_messages`.
- **Agent stops with empty text** → `max_tokens=4096` may be too low. Bump it to 8192.
- **Agent loops forever / hits max_turns** → the system prompt is too vague about when to stop. Add: "Once you have searched, scored, and written the report, stop."
- **0 filings found** → the agent isn't trying name variations. Add to the prompt: "Try ACME, ACME CORP, ACME CORPORATION, and DBA forms."

Step 4: Add Guardrails Manually

`30 min` `guardrails.py`

What & Why: A loop that does whatever Claude asks is not safe to ship. Production agents need at minimum: input validation (reject overly broad queries like `search_filings("a")`), PII redaction, a cost cap (kill the run after N tokens), and a circuit breaker (after K consecutive failures, stop and alert). You write all four by hand, sprinkled into the loop.

GUARDRAILS.PY

GUARDRAILS.PY

```
IMPORT re

SSN_RE = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
PHONE_RE = re.compile(r"\b\d{3}-\d{3}-\d{4}\b")
COST_LIMIT_TOKENS = 50_000
CIRCUIT_FAIL_THRESHOLD = 3

FUNCTION validate_input(tool_name, args):
    IF tool_name == "search_filings":
        q = args.get("debtor_name", "")
        IF len(q.strip()) < 3:
            RAISE ValueError("Query too broad: debtor_name must be ⩾3 characters")

FUNCTION redact_pii(payload):

    # … (full source in the desktop HTML) …

        self.failures = 0
    FUNCTION record(self, ok):
        IF ok= 0
        ELSE:  self.failures += 1
        IF self.failures ⩾ CIRCUIT_FAIL_THRESHOLD:
            RAISE RuntimeError("Circuit breaker tripped — aborting")
```

Now wire the guardrails into `_run_messages` in `agent.py`. This is the modified loop — replace your existing `_run_messages` function with the version below. New lines are flagged with `# NEW` comments.

AGENT.PY (REPLACE _RUN_MESSAGES)

AGENT.PY (REPLACE _RUN_MESSAGES)

```
# Add to the imports at the top of agent.py:
FROM guardrails import (validate_input, redact_pii,
                        CircuitBreaker, COST_LIMIT_TOKENS)   # NEW

_breaker = CircuitBreaker()        # NEW — module-level instance

FUNCTION _run_messages(messages):
    total_tokens = 0               # NEW — cost cap counter
    FOR turn IN range(MAX_TURNS):
        response = client.messages.create(
            model=MODEL, max_tokens=4096, system=SYSTEM,
            tools=TOOLS, messages=messages,
        )
        total_tokens += (response.usage.input_tokens

    # … (full source in the desktop HTML) …

            messages.append({"role": "user", "content": tool_results})
            CONTINUE

        RAISE RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")

    RAISE RuntimeError(f"Agent exceeded {MAX_TURNS} turns without finishing")
```

Run: `python agent.py` (no behavior change yet on a clean Acme query — guardrails only fire on bad inputs)

Test that the guardrails actually fire. Run this one-liner that calls the agent with a deliberately-broken question and confirms `validate_input` rejects it:

```
python -c "from agent import run_agent; print(run_agent('Search for company A'))"
# Expected: agent attempts search_filings(debtor_name='A'), validate_input
# raises ValueError, the error is fed back to Claude as is_error: true,
# and Claude reformulates with a longer fragment.
```

✅ **Checkpoint — Step 4**

The clean Acme query still works (same narrative as Step 3). The "company A" query gets rejected at `validate_input`, the error flows back to Claude, and Claude either reformulates or apologizes. If you see `RuntimeError: Cost cap exceeded`, your loop is unbounded — check that `total_tokens` is being checked after every `messages.create`.

Troubleshooting

- `ImportError: cannot import name 'validate_input' from 'guardrails'` → you didn't save `guardrails.py` in the project folder, OR the function name has a typo.
- **Circuit breaker trips on first call** → you wired `_breaker.record(ok=False)` on every tool call instead of only in the except. Move it back to the except branch.
- **Cost cap fires immediately** → `COST_LIMIT_TOKENS = 50_000` is for the whole run; if you set it to 50 by mistake it'll fire after the first response.

ITER-1 REALITY CHECK

You are now copy-pasting the same four guardrail calls into every tool execution path. This is exactly the boilerplate that hooks (Iter 2) eliminate. Notice how it feels — this annoyance is what motivates the next iteration.

Step 5: Add Logging Manually

`15 min` `audit.py + agent.py wiring`

What & Why: Auditability is non-negotiable for production agents. Every tool call needs a timestamped record: which tool, which inputs (redacted), the result summary, and the token count. Write a small `append_audit()` function in its own file, then call it from `_run_messages` after every successful tool execution.

Create a new file `audit.py` in the project folder:

AUDIT.PY

AUDIT.PY

```
IMPORT json, datetime
FROM guardrails import redact_pii

FUNCTION append_audit(tool_name, args, result, tokens,
                      case_id= "default"):
    rec = {
        "ts": datetime.datetime.utcnow().isoformat() + "Z",
        "case_id": case_id,
        "tool": tool_name,
        "input_redacted": redact_pii(json.dumps(args, default=str)),
        "output_summary": redact_pii(str(result))[:200],
        "tokens": tokens,
    }
    WITH open("audit_log.jsonl", "a") as f:
        f.write(json.dumps(rec) + "\n")
```

Now wire it into `_run_messages`. Add the import and the `append_audit` call right after the successful `execute_tool`:

AGENT.PY (ADDITIONS)

AGENT.PY (ADDITIONS)

```
# Add to the imports at the top of agent.py:
FROM audit import append_audit                    # NEW

# Inside _run_messages, in the try-block right after `result = execute_tool(...)`:
                        result = execute_tool(block.name, block.input)
                        _breaker.record(ok=True)
                        append_audit(                              # NEW
                            tool_name=block.name,
                            args=block.input,
                            result=result,
                            tokens=total_tokens,
                            case_id=messages[0]["content"][:40] IF messages else "default",
                        )
                        content = redact_pii(json.dumps(result, default=str))
                        # ... rest of the try-block unchanged ...
```

Run: `python agent.py`

Then inspect the audit file:

```
cat audit_log.jsonl   # macOS/Linux
type audit_log.jsonl  # Windows cmd
Get-Content audit_log.jsonl  # Windows PowerShell
```

Expected output (one JSON object per tool call — you should see roughly 4–6 lines):


```
{"ts": "2026-05-09T12:01:14Z", "case_id": "Assess the delinquency risk for Acme", "tool":
"search_filings", "input_redacted": "{\"debtor_name\": \"Acme Corporation\"}", "output_summary": "
[{\"filing_number\": \"NY-2024-0847\", ...}]", "tokens": 1842}
{"ts": "2026-05-09T12:01:18Z", "case_id": "Assess the delinquency risk for Acme", "tool":
"search_filings", "input_redacted": "{\"debtor_name\": \"ACME CORP\"}", "output_summary": "
[{\"filing_number\": \"NY-2024-0847\", ...}]", "tokens": 3021}
{"ts": "2026-05-09T12:01:22Z", "case_id": "Assess the delinquency risk for Acme", "tool":
"predict_delinquency", "input_redacted": "{\"active_filing_count\": 9, ...}", "output_summary": "
{\"probability\": 0.95, \"prediction\": \"HIGH\"}", "tokens": 4380}
```


✅ Checkpoint — Step 5

You should see one JSON object per line in `audit_log.jsonl`. If the file is missing, you ran the agent from the wrong directory; `cd` into the project folder. If the records are unreadable on one line, you forgot the `+ "\n"` in the `f.write` call.

Troubleshooting

- `ImportError: No module named 'audit'` → `audit.py` is in the wrong folder. Move it next to `agent.py`.

- **audit_log.jsonl is empty** → either the agent didn't make any tool calls (rare — check the system prompt) or your `append_audit` call is in the wrong place. It must be inside the try-block, after `execute_tool`, before the append to `tool_results`.
- **JSON parse errors when re-reading the log** → missing newline. Confirm `f.write(json.dumps(rec) + "\n")`.

Step 6: Multi-Turn Conversation

`15 min` `session.py`

What & Why: Real users ask follow-ups. "What about Pinnacle?" should reuse the analyst persona without re-asking the system prompt and without losing the prior conversation. Iter 1 implements multi-turn by maintaining a per-session messages list, appending the new user message, and reusing the same `_run_messages` helper from `agent.py`. A sliding window keeps the list from growing forever.

Create a new file `session.py` in the project folder:

SESSION.PY

SESSION.PY

```
FROM agent import _run_messages

SESSIONS, list] = {}   # session_id → messages list
WINDOW = 20                      # keep the last N messages

FUNCTION chat(session_id, user_msg):

    msgs = SESSIONS.setdefault(session_id, [])
    msgs.append({"role": "user", "content": user_msg})
    answer, msgs = _run_messages(msgs)
    # Sliding window= msgs[-WINDOW:]
    RETURN answer
```

Try it — multi-turn from the Python REPL:

```
python -c "
FROM session import chat
print(chat('case-001', 'What is the lien exposure for Acme Corporation?'))
print('---')
print(chat('case-001', 'Now compare to Pinnacle Industries.'))
"
```

Expected behavior: the second call refers to Pinnacle *in contrast to Acme* — only possible if the session retained the first turn. If you instead get a generic "Pinnacle Industries has 2 active filings..." that doesn't reference Acme, the session isn't carrying over.

✅ Checkpoint — Step 6

The second call's answer mentions Acme by name (proving the prior context survived). Try a third call: `chat('case-001', 'Which one is higher risk?')` — the agent should answer with reference to both. If it says "I don't know which two you mean", your `SESSIONS` dict isn't persisting across calls.

Troubleshooting

- `ImportError: cannot import name '_run_messages' from 'agent'` → you skipped Step 3's refactor of `agent.py`. Go back and split the loop into `_run_messages` + `run_agent`.
- **Each call starts fresh** → `SESSIONS` is a *module-level* dict. If you import `session` separately each time (e.g., from a fresh subprocess), the dict resets. For a real server, use a database or Redis instead of an in-memory dict.
- **Window slices off the wrong end** → `msgs[-WINDOW:]` keeps the LAST N messages (recent), not the first N. If your follow-ups lose context after many turns, the window is too small — bump `WINDOW` to 40.

Step 7: Deploy as FastAPI + Docker

`20 min` `server.py + Dockerfile + requirements.txt`

What & Why: Wrap the agent in a small HTTP API and a container so it is reachable from anywhere. Same Tier-1 deployment as M22B; the agent code does not care that it is in a container. We need three new files: `server.py` (FastAPI handlers), `Dockerfile` (build recipe), and `requirements.txt` (pinned dependencies the Dockerfile installs).

Create three files at the project root.

SERVER.PY · DOCKERFILE · REQUIREMENTS.TXT

SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run_agent
FROM session import chat as session_chat

app = FastAPI(title="UCC Risk Analyzer (Iter 1)")

CLASS Q(BaseModel):
    question: str

CLASS C(BaseModel):
    session_id: str
    message: str


    # … (full source in the desktop HTML) …


FUNCTION chat_ep(c):
    TRY:
        RETURN {"answer": session_chat(c.session_id, c.message)}
    CATCH:
        RAISE HTTPException(500, str(e))
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

REQUIREMENTS.TXT

```
anthropic ⩾ 0.40
fastapi ⩾ 0.110
uvicorn ⩾ 0.27
pydantic ⩾ 2.0
```

Run locally first (no Docker):

```
uvicorn server:app --reload --port 8000
```

In another terminal:

```
curl localhost:8000/health
# Expected: {"status":"ok","iter":1}

curl -X POST localhost:8000/query \
     -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}'
# Expected: {"answer":"Acme Corporation has 9 active UCC filings ..."}
```

Then build and run the Docker container:

```
docker build -t iter1-c .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter1-c
# In another terminal:
curl localhost:8000/health
```

Troubleshooting

- `docker: command not found` → install Docker Desktop and confirm `docker --version` works.
- `OSError: [Errno 98] Address already in use` → port 8000 is taken. Use `--port 8001` for uvicorn or `-p 8001:8000` for docker.
- **Container starts but /query returns 500 with "ANTHROPIC_API_KEY not set"** → you forgot the `-e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY` flag on docker run.
- **Container rebuild is slow** → expected. Each build re-installs dependencies. Use `--reload` with uvicorn for dev work; only build the image when you ship.

Iteration 1 Complete — End-to-End Verification

You should now have **8 files** in your project folder:

```
agent-iter1-raw/
├── agent.py            # the loop + _run_messages + run_agent
├── tools.py            # 3 tool schemas + execute_tool dispatcher
├── mock_data.py        # 12 UCC filings (9 Acme + 2 Pinnacle + 1 Sunrise)
├── delinquency_model.pkl   # sklearn RandomForest from Step 1
├── guardrails.py       # validate_input + redact_pii + CircuitBreaker
├── audit.py            # append_audit writer
├── session.py          # multi-turn chat() over _run_messages
├── server.py           # FastAPI handlers
├── Dockerfile          # python:3.11-slim build recipe
└── requirements.txt    # pinned deps
```

Run the full Iter-1 acceptance test:

```
# 1. Single-shot risk analysis
curl -s -X POST localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}' | python -m json.tool

# 2. Multi-turn (session_id keeps context)
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Lien exposure for Acme?"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Now compare to Pinnacle."}' | python -m json.tool

# 3. Guardrails fire on a too-broad query
curl -s -X POST localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question":"Search for company A"}' | python -m json.tool

# 4. Inspect the audit log
docker exec $(docker ps -q --filter ancestor=iter1-c) cat audit_log.jsonl | head -10
```

You pass Iter 1 if: (1) `/health` returns `ok`; (2) `/query` returns a narrative mentioning 9 Acme filings and a HIGH score; (3) the second `/chat` call references Acme (proving multi-turn); (4) the broad query gets reformulated by the agent after the gate denies it; (5) `audit_log.jsonl` has redacted entries for every tool call.

ITERATION 1 METRICS

- **Files:** 10 (agent.py, tools.py, mock_data.py, delinquency_model.pkl, guardrails.py, audit.py, session.py, server.py, Dockerfile, requirements.txt)
- **Lines you wrote:** ~250 (count `find . -name "*.py" -exec wc -l {} + | tail -1`)
- **Time:** ~3 hours
- **Abstractions used:** none — just `anthropic` Messages API and FastAPI

Debugging in Iteration 1: Print Statements + Manual Inspection

When the agent gives wrong output in Iter 1, you debug like you debug any Python program: by reading your own code. There is no abstraction between you and Claude; you can print everything.

A. Add a debug_turn() helper

Drop this into `agent.py` and call it after each `messages.create`:

PYTHON

PYTHON

```
FUNCTION debug_turn(turn_num, response, messages):
    print(f"\n═══ TURN {turn_num} ═══")
    print(f"stop_reason: {response.stop_reason}")
    FOR block IN response.content═ "tool_use":
            print(f"  TOOL: {block.name}({block.input})")
        ELIF block.type ═ "text":
            print(f"  TEXT: {block.text[:100]}...")
    print(f"  Messages in history: {len(messages)}")
    print(f"  Tokens={response.usage.input_tokens} out={response.usage.output_tokens}")
```

B. Inspect messages list manually

The most common Iter-1 bug is malformed messages. Add `import json; print(json.dumps(messages, default=str, indent=2))` right before each `messages.create` call. You should see strict alternation: user → assistant → user (with tool_results) → assistant ... If you see two assistant turns in a row, that is your bug.

C. Common Iter-1 bugs and how to spot them

- **Wrong `tool_use_id` in tool_result** → API returns 400. Check the `id` on the `tool_use` block matches `tool_use_id` exactly.
- **Forgot to append the assistant turn** → API complains about message order. Always append `response.content` BEFORE handling tool calls.
- **Loop never stops** → `stop_reason` is `tool_use` every turn. Either Claude keeps asking for tools (system prompt is too vague) or you forgot to handle `end_turn`.
- **Loop stops too early** → Claude returned `end_turn` with empty text. Usually because `max_tokens` is too low — bump from 1024 to 4096.
- **Tool returns garbage** → You forgot `json.dumps(result)` — the API requires a string for `tool_result.content`, not a Python dict.
- **UCC-specific:** agent finds 4 Acme filings but should find 9. Almost always a case-sensitive match in `search_filings` or skipping the DBA variant. Add `.lower()` on both sides.

D. Debug exercise (do this before moving on)

In `agent.py`, change `"tool_use_id": block.id` to `"tool_use_id": "wrong-id"` and re-run. You should get a 400 error from the API mentioning the missing tool_use_id. Read the error message carefully — it says *which* id was expected. Restore the correct id and re-run. **This is the muscle memory you build in Iter 1 that lets you debug Iter 3 generated code later.**

SESSION 2

Iteration 2: Agent SDK + Claude Code

Now you let the SDK run the loop, hooks handle the guardrails, sessions handle multi-turn, and Claude Code does most of the typing. Same agent. Half the lines. Different debugging.

`~2 hours` `~120 lines` `8 files` `SDK + hooks + sessions`

Step 8: Create CLAUDE.md via Claude Code

`10 min` `CLAUDE.md`

What & Why: CLAUDE.md is the project memory file Claude Code reads at every prompt. It tells Claude Code your conventions: where files live, what dependencies you use, what the system prompt should be, what tests to run. Without CLAUDE.md, every prompt becomes a re-explanation of the project.

CLAUDE CODE · CLAUDE.MD

CLAUDE CODE

```
mkdir agent-iter2-sdk && cd agent-iter2-sdk
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "claude-agent-sdk≥0.2" "scikit-learn≥1.3" "fastapi≥0.110" "uvicorn≥0.27"
npm i -g @anthropic-ai/claude-code   # if not already installed
claude
> /init
```

CLAUDE.MD

```
# Agent: UCC Filing Risk Analyzer (Iteration 2)

## Stack
- Python 3.11+
- `claude-agent-sdk` (the official Agent SDK — NOT a wrapper around `client.messages.create()`)
- scikit-learn for delinquency_model.pkl
- FastAPI + Docker for deployment
- Mock data in mock_data.py (12 UCC filings)

## File Layout
- agent.py            — query() entry point + @tool-decorated tools + create_sdk_mcp_server
- hooks/              — PreToolUse + PostToolUse hook scripts (referenced from .claude/settings.json)
- .claude/settings.json — hooks registration
- .claude/agents/     — subagent definitions (optional)
- sessions.py         — multi-turn session management
- server.py           — FastAPI wrapper

## System Prompt
You are a UCC filing risk analyst. Search thoroughly using name variations
including abbreviations and DBAs, gather statistics, run the delinquency
model, examine riskiest filings, write a narrative report citing evidence.
```

Step 9: Build Agent with @tool Decorators (claude-agent-sdk)

`15 min` `agent.py`

What & Why: The `claude-agent-sdk` lets you define tools as `@tool`-decorated async functions registered with an in-process MCP server. `query()` drives the loop for you — no `while True`, no `messages.append`, no `stop_reason` checks. The SDK is a real package; do NOT simulate it with `client.messages.create()`.

WHAT THE REAL SDK LOOKS LIKE

If you've used `client.messages.create()` in Iter 1, you might expect the SDK to be a thin `Agent` class wrapping it. It is not. The SDK is built around **MCP tools** + **an async** `query()` **generator** + **options/hooks via** `ClaudeAgentOptions`. Tools return `{"content": [{"type": "text", "text": ...}]}` (MCP shape), not bare Python values.

CLAUDE CODE PROMPT · AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

CLAUDE CODE PROMPT

```
> Create agent.py using `claude-agent-sdk`. Define three UCC tools as
> @tool-decorated functions: search_filings(debtor_name, state),
> get_filing_details(filing_number), predict_delinquency(...6 features).
> Wire them into a create_sdk_mcp_server, build ClaudeAgentOptions with the
> system prompt from CLAUDE.md, and expose an run(question) entry
> point that drives query() and concatenates AssistantMessage text.
```

AGENT.PY (PYTHON)

```python
IMPORT json, pickle
FROM claude_agent_sdk import (
    query, tool, create_sdk_mcp_server,
    ClaudeAgentOptions, AssistantMessage,
)
FROM mock_data import FILINGS

_MODEL = pickle.load(open("delinquency_model.pkl", "rb"))

@tool("search_filings",
      "Search UCC filings by debtor name (case-insensitive partial match).",
      {"debtor_name": str, "state": str})
FUNCTION search_filings(args):
    q = args["debtor_name"].lower()

    # … (full source in the desktop HTML) …

    FOR msg IN query(prompt=question, options=OPTIONS):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None):
                    parts.append(block.text)
    RETURN "\n".join(parts)
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — @anthropic-ai/claude-agent-sdk version
IMPORT { query, tool, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
IMPORT { z } from "zod";
IMPORT { readFileSync } from "fs";
IMPORT { FILINGS } from "./mock_data.js";
// (load delinquency model via your preferred Node ML lib, e.g. onnxruntime-node)

const searchFilings = tool(
  "search_filings",
  "Search UCC filings by debtor name (case-insensitive partial match).",
  { debtor_name: z.string(), state: z.string().optional() },
  (args) ⇒ {
    const q = args.debtor_name.toLowerCase();
    const st = (args.state ?? "").toUpperCase();

    # … (full source in the desktop HTML) …

        IF ("text" in block) parts.push(block.text);
      }
    }
  }
  RETURN parts.join("\n");
}
```

WHAT JUST DISAPPEARED

You no longer write the message loop, the stop_reason check, the tool_result append, or JSON schema dicts. The 90-line raw loop from Iter 1 collapsed to one `async for msg in query(...)` (Python) / `for await` (TS). The MCP server is created in 4 lines. **Same behavior, ~70 lines instead of ~140.**

Troubleshooting

- `ModuleNotFoundError: No module named 'claude_agent_sdk'` → `pip install "claude-agent-sdk ⩾ 0.2"` in your venv.

- `ImportError: cannot import name 'Agent' from 'anthropic'` → you're trying the old fictional API. The real SDK is `claude_agent_sdk`, not `anthropic.Agent`.
- Tool returns "Tool not allowed" → add `"mcp__<server>__<tool>"` to `allowed_tools`.

Step 10: Add Hooks via .claude/settings.json + HookMatcher

`20 min` `hooks/*.py + .claude/settings.json`

What & Why: The SDK supports two hook surfaces: (a) **file-based hooks** in `.claude/settings.json` that shell out to scripts (production-friendly, language-agnostic), and (b) **in-process hooks** via `HookMatcher` in `ClaudeAgentOptions(hooks={...})`. We use (a) for redaction/audit (matches the Tier 3 cert pattern) and (b) for in-process validation that needs to raise.

.CLAUDE/SETTINGS.JSON · HOOKS/REDACT_PII.PY · IN-PROCESS VALIDATION

.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "mcp__ucc__search_filings",
        "command": "python hooks/log_call.py" }
    ],
    "PostToolUse": [
      { "matcher": "*", "command": "python hooks/redact_pii.py" },
      { "matcher": "*", "command": "python hooks/audit.py" }
    ]
  }
}
```

HOOKS/REDACT_PII.PY

```
IMPORT sys, json, re

SSN_RE = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
PHONE_RE = re.compile(r"\b\d{3}-\d{3}-\d{4}\b")

payload = json.load(sys.stdin)
text = json.dumps(payload.get("tool_result"), default=str)
text = SSN_RE.sub("[SSN_REDACTED]", text)
text = PHONE_RE.sub("[PHONE_REDACTED]", text)
payload["tool_result"] = json.loads(text)
json.dump(payload, sys.stdout)
```

IN-PROCESS VALIDATION

```
FROM claude_agent_sdk import HookMatcher

FUNCTION validate_query(input_data, tool_use_id, context):
    IF input_data.get("tool_name", "").endswith("search_filings"):
        q = (input_data.get("tool_input", {}).get("debtor_name") or "").strip()
        IF len(q) < 3:
            RETURN {"hookSpecificOutput": {"hookEventName": "PreToolUse",
                                           "permissionDecision": "deny",
                                           "permissionDecisionReason": "Query too broad"}}
    RETURN {}

# Then update OPTIONS in agent.py= ClaudeAgentOptions(
    # ... existing fields ...
    hooks={"PreToolUse": [HookMatcher(matcher="mcp__ucc__search_filings",
                                      hooks=[validate_query])]},
)
```

Create the supporting hook scripts — both follow the same stdin/stdout pattern as `redact_pii.py`. Save these in `hooks/`:

HOOKS/LOG_CALL.PY · HOOKS/AUDIT.PY

HOOKS/LOG_CALL.PY

```
IMPORT sys, json, datetime

payload = json.load(sys.stdin)
ts = datetime.datetime.utcnow().isoformat() + "Z"
name = payload.get("tool_name", "?")
args = payload.get("tool_input", {})
print(f"[{ts}] PRE  {name}({args})", file=sys.stderr)
json.dump(payload, sys.stdout)   # pass-through — do NOT modify
```

HOOKS/AUDIT.PY

```
IMPORT sys, json, datetime

payload = json.load(sys.stdin)
rec = {
    "ts": datetime.datetime.utcnow().isoformat() + "Z",
    "tool": payload.get("tool_name"),
    "input": payload.get("tool_input"),
    "output_summary": str(payload.get("tool_result"))[:200],
}
WITH open("audit_log.jsonl", "a") as f:
    f.write(json.dumps(rec, default=str) + "\n")
json.dump(payload, sys.stdout)   # pass-through
```

Debug each hook standalone before wiring them up:

```
# Smoke-test the redactor (should replace the SSN):
echo '{"tool_name":"search_filings","tool_result":"customer SSN 123-45-6789"}' \
  | python hooks/redact_pii.py

# Smoke-test the audit writer (should append a line):
echo '{"tool_name":"search_filings","tool_input":{"debtor_name":"Acme"},"tool_result":"ok"}' \
  | python hooks/audit.py
cat audit_log.jsonl  # should show the new entry
```

Then run the agent end-to-end:

```
python -c "import asyncio, agent; print(asyncio.run(agent.run('Risk for Acme Corporation?')))"
```

✅ **Checkpoint — Step 10**

You should see (1) `[timestamp] PRE search_filings(...)` log lines on stderr from `log_call.py`, (2) the agent's narrative report on stdout, (3) one line per tool call in `audit_log.jsonl`. Try a 1-character query — the in-process `HookMatcher` validator should deny it with "Query too broad".

Troubleshooting

- **Hooks don't run at all** → the SDK looks for `.claude/settings.json` in the current working directory. Run from the project root.
- **Hook script crashes on JSON parse** → you forgot to `json.dump(payload, sys.stdout)` as the LAST line. The SDK reads the script's stdout and parses it back as JSON; an empty or malformed stdout breaks the pipe.
- **Audit log missing entries** → the matcher is too narrow. `"matcher": "*"` matches every tool call; `"matcher": "search_filings"` only matches that one tool name.
- **In-process validator never fires** → check that `hooks={"PreToolUse": [HookMatcher(...)]}` is a kwarg on `ClaudeAgentOptions`, not a separate variable.

Step 11: Sessions — Multi-Turn + Resume

`15 min` `sessions.py`

What & Why: The SDK supports session continuation via the `resume` option (or `continue_conversation=True` for the most recent), and what-if branching via copying a session id. Iter 1's `SESSIONS` dict was a hand-rolled approximation; here we keep a thin in-memory map of `session_id → last_resume_token` and let the SDK handle the rest.

SESSIONS.PY

SESSIONS.PY

```
FROM dataclasses import replace
FROM claude_agent_sdk import query, AssistantMessage
FROM agent import OPTIONS

SESSIONS, str] = {}   # case_id → resume token (session_id)

FUNCTION _drive(prompt, options):
    parts, last_session_id = [], None
    FOR msg IN query(prompt=prompt, options=options):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None):
                    parts.append(block.text)
        # The SDK emits a session_id on the result message at the end.

    # … (full source in the desktop HTML) …

FUNCTION what_if(session_id, hypothetical):

    resume = SESSIONS.get(session_id)
    options = replace(OPTIONS, resume=resume) IF resume else OPTIONS
    text, _ = _drive(hypothetical, options)
    RETURN text
```

Try it — multi-turn + fork demo:

```
python -c "
IMPORT asyncio
FROM sessions import chat, what_if

FUNCTION main():
    # Turn 1: establish context
    print('T1, chat('case-001', 'What is the lien exposure for Acme Corporation?'))
    # Turn 2: builds on T1
    print('T2, chat('case-001', 'Now compare to Pinnacle Industries.'))
    # Fork: hypothetical that does NOT pollute the main session
    print('FORK, what_if('case-001', 'What IF Acme files a UCC-3 termination on NY-2024-0848?'))
    # Turn 3 on the original session: the fork's hypothetical was NOT seen
    print('T3, chat('case-001', 'Summarize Acme vs Pinnacle in one sentence.'))

asyncio.run(main())
"
```

✅ **Checkpoint — Step 11**

T2 should reference Acme by name (proving session persistence). T3 should NOT mention the hypothetical UCC-3 termination from FORK (proving fork isolation). If T3 includes the termination scenario, your `what_if` is updating `SESSIONS[session_id]` when it shouldn't.

Troubleshooting

- `TypeError: replace() got an unexpected keyword argument 'resume'` → `ClaudeAgentOptions` in your installed SDK version doesn't support `resume=`. Upgrade with `pip install -U "claude-agent-sdk ⩾ 0.2"`.
- **session_id is always None on first call** → expected. The SDK only emits `session_id` on the result message AFTER the agent finishes. The first turn has no resume token; subsequent turns do.
- **Fork still updates the main session** → check that `what_if` uses `text, _ = await _drive(...)` (discarding the new sid) rather than `text, sid = await _drive(...)` followed by an assignment.

Step 12: Slash Commands

`15 min` `.claude/commands/*.md (3 files)`

What & Why: Claude Code slash commands let you run reusable workflows from inside the IDE. `/run-agent` calls the agent on a question; `/test-agent` runs the unit test suite; `/eval-agent` runs the 10-question UCC eval set and reports per-scenario scoring.

Create 3 files in `.claude/commands/` :

RUN-AGENT.MD · TEST-AGENT.MD · EVAL-AGENT.MD

RUN-AGENT.MD

```
---
description: Run the UCC Risk Analyzer agent on a question
argument-hint: [question]
---
Run `python -c "import asyncio, agent; print(asyncio.run(agent.run('$ARGUMENTS')))"`.
If --verbose is in $ARGUMENTS, set AGENT_VERBOSE=1 (read by hooks/log_call.py).
Print the final report and the token + cost summary from query()'s ResultMessage.
```

TEST-AGENT.MD

```
---
description: Run the unit test suite for the UCC agent
---
Run `pytest tests/ -v`. The test suite covers tools, hooks, sessions, and the
end-to-end agent (must find all 9 Acme filings including the DBA variant).
Report each test's pass/fail and a summary at the end. If any tests fail,
read the spec at `spec/agent-spec.md` (in Iter 3) to see expected behavior,
then fix the implementation.
```

EVAL-AGENT.MD

```
---
description: Run the 10-question evaluation suite
---
Read `test_scenarios.json` (10 question/expected-output pairs). For each
question, call `agent.run()`, score the response on a rubric: did it find
the right entities? did it return the right risk level? did it cite specific
filings? Report per-scenario score (0-5) and an overall percentage.
```

Try it from inside Claude Code:

```
claude
> /run-agent What is the lien exposure for Acme Corporation?
> /test-agent
> /eval-agent
```

✅ **Checkpoint — Step 12**

When you type `/` in Claude Code, the three commands appear in the autocomplete. Running `/run-agent` with a UCC question produces the narrative report. `/test-agent` requires you to first create a `tests/` folder — in Iter 3 the spec generates these for you.

Troubleshooting

- **Slash commands don't appear in autocomplete** → the directory must be exactly `.claude/commands/` at the project root. Restart Claude Code after creating new files.
- `$ARGUMENTS` isn't substituted → this is a Claude Code template variable; it works inside the slash command, not in plain bash. If you're testing the underlying Python directly, hardcode the question.

Step 13: Deploy via Claude Code

`15 min` `server.py + Dockerfile`

What & Why: Same FastAPI + Docker pattern as Iter 1, but you ask Claude Code to write it.

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Create server.py and Dockerfile. Endpoints: GET /health, POST /query
> (single-shot), POST /chat (session_id + message). The endpoints are async
> FastAPI handlers that await agent.run() and sessions.chat() respectively
> (both are async coroutines from claude-agent-sdk). Dockerfile based on
> python:3.11-slim, install claude-agent-sdk + dependencies, expose 8000,
> uvicorn entrypoint. Mount .claude/ into the container so settings.json
> and hook scripts are picked up.
```

What Claude Code produces (review before saving):

SERVER.PY · DOCKERFILE · REQUIREMENTS.TXT

SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run as agent_run
FROM sessions import chat as session_chat

app = FastAPI(title="UCC Risk Analyzer (Iter 2 — SDK)")

CLASS Q(BaseModel): question: str
CLASS C(BaseModel): session_id: str; message: str

FUNCTION health(): return {"status": "ok", "iter": 2}

FUNCTION query(q):
    TRY: return {"answer": agent_run(q.question)}
    CATCH: raise HTTPException(500, str(e))

FUNCTION chat_ep(c):
    TRY: return {"answer": session_chat(c.session_id, c.message)}
    CATCH: raise HTTPException(500, str(e))
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# .claude/settings.json + hooks/ must be in the build context for hooks to fire
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

REQUIREMENTS.TXT

```
claude-agent-sdk ⩾ 0.2
fastapi ⩾ 0.110
uvicorn ⩾ 0.27
pydantic ⩾ 2.0
```

Run locally first:

```
uvicorn server:app --reload --port 8000
# In another terminal:
curl localhost:8000/health
# Expected: {"status":"ok","iter":2}

curl -X POST localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}'
```

Then build and run with Docker:

```
docker build -t iter2-c .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter2-c
```

Troubleshooting

- **Container starts but hooks don't fire** → `.claude/` wasn't copied into the image. Confirm `COPY . .` includes the `.claude/` folder; some `.dockerignore` setups exclude dotfiles.

- `RuntimeError: This event loop is already running` → the FastAPI handlers must be `async def`, not plain `def` (they're awaiting the SDK's async `query()`).
- **Slow build** → pin versions in requirements.txt and add a `.dockerignore` with `venv/`, `__pycache__/`, `*.pyc` to skip unnecessary copies.

🥕 Iteration 2 Complete — End-to-End Verification

You should now have **~13 files** in your project:

```
agent-iter2-sdk/
├── CLAUDE.md
├── agent.py            # query() + 3 @tool functions + create_sdk_mcp_server
├── sessions.py         # chat() and what_if() over SDK resume tokens
├── mock_data.py        # same 12 records as Iter 1
├── delinquency_model.pkl
├── .claude/
│   ├── settings.json    # PreToolUse + PostToolUse matchers
│   ├── commands/run-agent.md, test-agent.md, eval-agent.md
├── hooks/
│   ├── log_call.py, redact_pii.py, audit.py
├── server.py | Dockerfile | requirements.txt
```

Run the same Iter-1 acceptance test against the Iter-2 deployment — the curl shape and outputs are identical:

```
# Same 4 acceptance curls as Iter 1, but pointing at the Iter-2 container.
# Outputs should be functionally identical to Iter 1 (same 9 Acme filings, HIGH score).
curl -s -X POST localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}' | python -m json.tool

# Multi-turn via the SDK's resume token (different mechanism than Iter 1's transcript)
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Lien exposure for Acme?"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Now compare to Pinnacle."}' | python -m json.tool

# Hook-driven audit (instead of Iter-1's hand-rolled append_audit call)
docker exec $(docker ps -q --filter ancestor=iter2-c) cat audit_log.jsonl | head -10
```

You pass Iter 2 if: all 4 outputs are functionally equivalent to Iter 1, but you wrote ~120 lines instead of ~250. The agent's *observable behavior* is unchanged; the implementation underneath uses the SDK + hooks + Claude Code.

ITERATION 2 METRICS

- **Files:** ~13 (CLAUDE.md, agent.py, sessions.py, mock_data.py, delinquency_model.pkl, .claude/settings.json, .claude/commands/×3, hooks/×3, server.py, Dockerfile, requirements.txt)
- **Lines you wrote:** ~120 (vs Iter 1's ~250)
- **Time:** ~2 hours (vs Iter 1's ~3)
- **Abstractions used:** `claude-agent-sdk` (query / @tool / create_sdk_mcp_server / ClaudeAgentOptions / HookMatcher / PermissionResultDeny) + Claude Code slash commands

Debugging in Iteration 2: Hooks + Console Web UI + Langfuse

The SDK abstracts the loop — you cannot drop a print statement inside it. You debug from the outside instead.

A. Hooks as Debug Probes

Hooks fire at every tool call. The simplest debug surface is a stdin-stdout script registered in `.claude/settings.json` that pretty-prints to stderr (so it doesn't pollute the hook payload):

HOOKS/DEBUG.PY · .CLAUDE/SETTINGS.JSON (DEBUG)

HOOKS/DEBUG.PY

```
IMPORT sys, json
payload = json.load(sys.stdin)
print(f"[HOOK {payload.get('hook_event_name','?')}] "
      f"{payload.get('tool_name','?')}: "
      f"{json.dumps(payload.get('tool_input', payload.get('tool_result')))[:200]}",
      file=sys.stderr)
json.dump(payload, sys.stdout)   # pass-through
```

.CLAUDE/SETTINGS.JSON (DEBUG)

```
{
  "hooks": {
    "PreToolUse":  [{ "matcher": "*", "command": "python hooks/debug.py" }],
    "PostToolUse": [{ "matcher": "*", "command": "python hooks/debug.py" }]
  }
}
```

BETTER than Iter-1 print statements because hooks are modular — swap `.claude/settings.json` back to the production version when you're done debugging and the agent code stays untouched.

B. Anthropic Console Web UI

The Anthropic Console (console.anthropic.com) shows every API call your agent made.

Worked example: Agent calls `search_filings("acme")` but gets 0 results when 9 are expected. Console → Logs → failed call → tool_use block shows Claude sent `"acme"` (lowercase) but the generated `search_filings` compares without lowercasing the data side. Fix: `q in f["debtor_name"].lower()`. Re-run; Console shows the fix working. Total time to diagnose: ~3 minutes vs ~15 in Iter 1.

C. Langfuse Traces (if instrumented in M19)

If you wired Langfuse in M19, every agent run is a waterfall trace. Each tool call is a span with timing, input, output. Compare traces across runs — "why did today's run take 12 seconds vs yesterday's 4?" gets answered by reading the spans.

D. /run-agent --verbose

The slash command can include a verbose flag that turns on the debug hooks for one run. Output: every tool call, every hook fire, every token count, total cost, total time. Zero permanent code changes.

SESSION 3

Iteration 3: Spec-Driven

You stop writing code. You write a spec describing what the agent should do, then ask Claude Code to build it. ~18 files appear. You review, iterate on the spec, regenerate. The spec IS the documentation.

`~1 hour` `~100 lines of spec` `~18 generated files` `0 lines of agent code`

Step 14: Write agent-spec.md (12 sections)

`30 min` `agent-spec.md`

What & Why: The spec is your full design doc. Claude Code reads it and produces every file from it. Be specific in section 3 about parameter names and types — that is what generates the schemas.

AGENT-SPEC.MD (EXCERPT)

AGENT-SPEC.MD (EXCERPT)

```
# Agent Specification: UCC Filing Risk Analyzer

## 1. Overview
An agent that assesses delinquency risk for businesses by searching UCC
filings, running an ML model, and writing a narrative risk report.

## 2. Configuration
- Model: claude-sonnet-4-6
- Framework: claude-agent-sdk (Python). Tools registered via
  create_sdk_mcp_server. Driver: query() with ClaudeAgentOptions.
- Max turns: 10
- Max tokens per response: 4096

## 3. Tools (registered as MCP server "ucc")

    # … (full source in the desktop HTML) …

|— sessions.py                # resume-token sessions
|— mock_data.py | delinquency_model.pkl
|— hooks/{log_call,redact_pii,audit}.py
|— server.py | Dockerfile | docker-compose.yml | requirements.txt
|— tests/ x5
|— appendix/manual-loop.py     # Iter-1 reference, not for production
```

Step 15: One Command Generates Everything

10 min ~18 files appear

What & Why: Open Claude Code in the project folder. Issue ONE prompt that points at the spec and asks for a full build. Watch it work — this is the moment Iteration 3 earns its name.

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> /generate-from-spec spec/agent-spec.md

# (or, if /generate-from-spec is not installed:)
> Read spec/agent-spec.md and build the entire project. Create every file
> listed in section 12 (File Structure). Implement every tool, hook, test,
> and endpoint exactly as specified. Use the `claude-agent-sdk` package
> (NOT a fictional anthropic.Agent class). Tools must be @tool-decorated
> functions registered via create_sdk_mcp_server. The driver is
> query() + ClaudeAgentOptions. Hooks live in .claude/settings.json
> AND in OPTIONS.hooks (HookMatcher) per section 5. Generate realistic
> mock UCC filing data. Train and save the pickle model. Also generate
> appendix/manual-loop.py as an under-the-hood reference.
```

What you should see Claude Code create — expect a stream of file-creation messages over 5–10 minutes:

```
Reading spec/agent-spec.md ...
Created agent.py (78 lines)
Created tools.py (52 lines — 3 @tool functions + create_sdk_mcp_server)
Created hooks/log_call.py (12 lines)
Created hooks/redact_pii.py (14 lines)
Created hooks/audit.py (15 lines)
Created sessions.py (28 lines)
Created mock_data.py (62 lines — 12 UCC filings)
Created delinquency_model.pkl (binary, 28 KB — trained sklearn RandomForest)
Created server.py (24 lines)
Created Dockerfile (8 lines)
Created docker-compose.yml (10 lines)
Created requirements.txt (4 lines)
Created CLAUDE.md (35 lines)
Created .claude/settings.json (12 lines)
Created .claude/commands/run-agent.md, test-agent.md, eval-agent.md (3 files)
Created tests/test_tools.py, test_agent.py, test_hooks.py, test_sessions.py, test_api.py (5 files)
```

```
Created spec/test_scenarios.json (10 scenarios)
Created appendix/manual-loop.py (~80 lines — under-the-hood reference)
Total: 24 files, ~510 generated lines + 100 lines of spec you wrote.
```

Verify it actually works:

```
# From inside Claude Code:
> /test-agent

# Or from a regular shell:
pytest tests/ -v
```

Expected pytest output:

```
tests/test_tools.py::test_search_filings_acme_finds_9 PASSED
tests/test_tools.py::test_get_filing_details_returns_full_record PASSED
tests/test_tools.py::test_predict_delinquency_returns_probability PASSED
tests/test_agent.py::test_finds_all_acme_filings_including_dba PASSED
tests/test_agent.py::test_returns_high_risk_for_acme PASSED
tests/test_hooks.py::test_pii_redacted_from_audit_log PASSED
tests/test_hooks.py::test_short_query_denied_by_validator PASSED
tests/test_sessions.py::test_chat_persists_context PASSED
tests/test_sessions.py::test_what_if_does_not_pollute_main_session PASSED
tests/test_api.py::test_health_returns_ok PASSED
========================== 10 passed in 12.34s ============================
```

✅ **Checkpoint — Step 15**

All 10 tests pass on the first run. If `test_finds_all_acme_filings_including_dba` fails (got 5, expected 9), the generated `search_filings` probably skipped the DBA variant — the spec was ambiguous. See the debugging block below for how to fix it via spec edit (NOT by editing the generated code).

Troubleshooting

- **Claude Code asks clarifying questions instead of generating** → the spec has gaps. Either answer inline or update the spec and re-prompt with "Use the updated spec".
- **Some files are missing** → section 12 of the spec lists every expected file; if the generator missed one, point it out: "tests/test_hooks.py wasn't created — please add it per spec section 10".
- **Generated code uses fictional API** → re-issue the prompt and explicitly say "use the real `claude_agent_sdk` package — do NOT use anthropic.Agent or @agent.tool". Then ask it to compare the generated agent.py against spec section 2.

Step 16: Review & Iterate on the Spec

`15 min` `spec edits + targeted regen`

What & Why: The spec is the source of truth. When you want to add or change behavior, edit the spec FIRST, then ask Claude Code to regenerate the affected files. Don't edit generated code directly — the next regen would overwrite you.

Example: add a fourth tool `check_lapse_dates(months_ahead: int = 12)` that returns filings approaching lapse:

1. Edit `agent-spec.md` section 3 to add the new tool
2. Edit section 11 to add an eval scenario for it
3. In Claude Code: *"I added check_lapse_dates to agent-spec.md. Update tools.py, mock_data.py, add a test, and an eval scenario."*
4. Claude Code reads the spec diff and makes *targeted* changes — not a full regen
5. `/test-agent` → all green

Step 17: Deploy & Compare

`15 min` `same curl, same output`

What & Why: The Dockerfile and docker-compose.yml are already generated. Build, run, curl, compare output to Iter 1 and Iter 2 — the test of "spec-driven works" is that the agent's *observable behavior* matches the prior iterations.

SHELL

SHELL

```
# Build and run via docker-compose (generated by Claude Code)
docker compose up --build -d
docker compose ps   # confirm container is "Up"

# Health check
curl localhost:8000/health
# Expected: {"status":"ok","iter":3}

# Same query as Iter 1 and Iter 2
curl -X POST localhost:8000/query \
     -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}' | python -m json.tool
```

Cross-iteration diff — this is where the punchline lands:

```
# Run the same query against all three deployments and diff
# (assumes Iter 1 on :8001, Iter 2 on :8002, Iter 3 on :8003)
for port in 8001 8002 8003; do
  echo "═══ Iter on :$port ═══"
  curl -s -X POST localhost:$port/query \
       -H "Content-Type: application/json" \
       -d '{"question":"Risk for Acme Corporation?"}' \
    | python -c "import json, sys; d = json.load(sys.stdin); print(d['answer'][:300])"
  echo
done
```

Expected: all three responses mention 9 Acme filings, 4 states, the DBA variant, and HIGH risk. Wording will vary (Claude is non-deterministic), but the *facts* match.

Iteration 3 Complete — The Whole Capstone

You should have **~24 generated files** (Claude Code wrote them) plus your **~100-line spec**:

```
ucc-risk-agent/
├── spec/agent-spec.md          # YOU wrote this
├── spec/test_scenarios.json    # generated
├── CLAUDE.md                   # generated
├── .claude/settings.json       # generated
├── .claude/commands/ ×3        # generated
├── agent.py | tools.py | sessions.py    # generated (SDK)
├── hooks/ ×3                    # generated
├── mock_data.py | delinquency_model.pkl   # generated
├── tests/ ×5                    # generated
├── server.py | Dockerfile | docker-compose.yml | requirements.txt   # generated
└── appendix/manual-loop.py     # generated reference
```

Acceptance test — same shape as Iter 1 and Iter 2:

```
# 1. Eval suite passes
pytest tests/ -v
# Expected: 10 passed

# 2. End-to-end question on the deployed container
curl -s -X POST localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question":"Risk for Acme Corporation?"}' | python -m json.tool

# 3. Multi-turn (SDK resume tokens)
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Lien exposure for Acme?"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"session_id":"acceptance","message":"Now compare to Pinnacle."}' | python -m json.tool

# 4. Spec-vs-code drift check (the Iter-3 specific verification)
claude
> Read spec/agent-spec.md and compare to agent.py + hooks/. Report any drift.
# Expected: "No drift detected. Code matches spec section by section."
```

You pass Iter 3 if: (1) all 10 generated tests pass; (2) curl outputs match Iter 1/2 facts; (3) the spec-vs-code drift check returns "no drift". You wrote a 100-line spec; Claude Code wrote 510 lines of working code from it.

ITERATION 3 METRICS

- **Files:** ~24 (all generated by Claude Code from your spec)
- **Lines you wrote:** ~100 (the spec only)
- **Lines Claude Code wrote:** ~510 (and they're all readable, runnable, and pass the tests)
- **Time:** ~1 hour (~30 min writing the spec, ~10 min on the generation prompt, ~20 min reviewing + fixing)
- **Abstractions used:** spec format + `/generate-from-spec` + Claude Code + the same SDK runtime as Iter 2

Debugging in Iteration 3: Spec Comparison + Tests + Evals

When generated code has bugs, you do NOT debug it line by line. You debug at the spec level. The code is regenerable; the spec is not.

A. Spec vs Code Comparison

Most spec-driven bugs are spec/code drift. Ask Claude Code:

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Read agent-spec.md and compare it to the generated agent.py and tools.py.
> Report any deviations where the code does not match the spec.
```

Claude Code reads both and tells you exactly what diverged. The spec is the truth — if the code deviates, the code is wrong.

B. Test-Driven Debugging

The spec includes test definitions. When tests fail:

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Test test_finds_all_acme_filings failed: expected 9 filings, got 5.
> Read agent-spec.md section 3 (Tools) for the search_filings spec.
> Read the generated search_filings function. Find why it misses 4 filings
> (likely: case-sensitive match or skipping DBA variant). Fix it.
```

C. Eval-Driven Debugging

Run `/eval-agent`. Output: scenario 6 scored 2/5 — "agent did not try DBA variations." This is a behavior bug, not a code bug.

1. Open `agent-spec.md` section 4 (System Prompt)
2. Make it more explicit: "Always try at least 3 name variations including any known DBAs"
3. In Claude Code: *"I updated the system prompt. Regenerate only agent.py."*
4. `/eval-agent` again → scenario 6 now scores 5/5

D. Console + Langfuse (same as Iter 2)

Because the generated code uses the same Agent SDK and hooks as Iter 2, all the same Console Web UI and Langfuse debugging from Iter 2 still work.

THE DEBUGGING EVOLUTION SUMMARY

ITERATION	PRIMARY DEBUG METHOD	SECONDARY	SPEED TO FIX
1 Raw	print() in the loop	Manual message inspection	Slow (find the line)
2 SDK	Hooks + Console Web UI	Langfuse traces	Medium (modular probes)
3 Spec	Spec vs code comparison	Tests + evals + Console	Fast (Claude Code finds it)

The Comparison Table

Same UCC agent, same business question. Here is everything that changed:

METRIC	ITER 1: RAW API	ITER 2: AGENT SDK	ITER 3: SPEC-DRIVEN
Lines YOU wrote	~250	~120	~100 (spec only)
Time to build	~3 hours	~2 hours	~1 hour
Agent output	Baseline	Same	Same
Guardrails	Inline code in loop	Hooks (modular)	Hooks (generated)
Multi-turn	Manual history dict	SDK sessions	SDK sessions (generated)
Adding a new tool	Edit 3 files	One Claude Code prompt	Update spec, ask Claude Code to regen
Tests	Manual	Claude Code generated	Spec generates them
Documentation	Separate (or none)	CLAUDE.md	Spec IS the docs
Control over internals	Full	SDK-managed	Least direct (but reviewable)
Understanding needed	Every line	SDK abstractions	Architecture-level
Debugging	print() in loop	Hooks + Console Web UI + Langfuse	Spec comparison + tests + evals
Onboarding a new dev	Read 7 files	Read CLAUDE.md + 8 files	Read 1 spec file

KEY TAKEAWAY

All three produce the same UCC risk agent. The difference is what each iteration teaches you. Iteration 1 teaches WHAT an agent is. Iteration 2 teaches HOW to build efficiently. Iteration 3 teaches HOW production teams work. **You need all three.** Skip Iter 1 and you cannot debug Iter 3. Skip Iter 3 and you are 10x slower. Skip Iter 2 and you cannot understand the trade-off curve between them.